Table of Contents

Introduction **2**

Why Use .NET 10 for Web API? **3**

Setting up backend API **4**

Creating the Project *4*

Transitioning Code: MVC Monolith to Decoupled API *4*

Managing Dependencies (.csproj) *6*

`Program.cs` Pipeline Configuration *7*

Configuration Setup *8*

Test the api with scalar *9*

Why We Use DTOs Instead of Database Entities *10*

Create the Data Transfer Object (ItemDto) *11*

Create the API Controller *12*

Test the api endpint *13*

Alternative ways of testing the backend API **14**

Setting up debugging of the backend API **15**

Configuring Properties/launchSettings.json and dotnet watch *15*

Configure VS Code Debugger (`.vscode/`) *15*

Demo: API and Debugging

Author: Baifan Zhou
Detailed explanations are created with the help of ChatGPT/Gemini.

Introduction

In this demo, we will set up and debug the backend API based on .NET 10.0. We will cover the initial setup of the project, configuring it for debugging, creating the API controller, using Data Transfer Objects (DTOs) to simplify the response data, and testing the API.

Setting Up Backend API

- **Create a New Project:** Begin by creating a new .NET project using the `webapi` template with controller support (`--use-controllers`) and name it `api`.
- **Update csproj File:** Copy the required PackageReference entries from your previous project's .csproj file into the new api.csproj. Ensure you include packages necessary for testing with Scalar UI.
- **Configure Program.cs:** Update Program.cs to set up the web application, including services like Scalar and CORS. Configure logging and routing, and adjust settings for development mode (e.g., database seeding and Scalar UI).
- **Update appsettings.json:** Add the ConnectionStrings section to appsettings.json for database connectivity.

Creating API Controller

- **Define API Controller:** Update ItemController.cs to use the [ApiController] attribute and configure routes for API endpoints. Implement actions like ItemList to handle HTTP GET requests.

Using Data Transfer Objects (DTOs)

- **Create DTOs:** Define DTOs to tailor data formats and simplify the API responses. Create a folder named DTOs and add DTO classes that exclude complex relationships like navigation properties.

Testing the Backend API

- Use scalar
- Use terminal commands like curl
- Directly enter the API URL in the browser

Setting Up Debugging:

- **Configure launchSettings.json:** Set `launchUrl` to `scalar/v1` so `dotnet watch` automatically launches the browser to the API documentation dashboard.
- **Create launch.json and task.json:** Add these files in the .vscode folder, ensuring the correct paths to your api and myshop folders are specified.
- **Run Debugging:** Access the API in the browser

Why Use .NET 10 for Web API?

Before setting up the project, we choose .NET 10 for the backend API over other frameworks like Node.js (Express), Python (FastAPI), or Java (Spring Boot):

- **Ecosystem & Code Reuse:** Since the first half of the course uses .NET 10 MVC, using .NET 10 for the API allows reusing C# models, Data Access Layers (DAL), Entity Framework Core (DbContext), and validation logic directly, eliminating the need to learn a new programming language or ORM.
- **Industry-Leading Performance:** .NET 10 provides top-tier throughput, optimised memory management, and extremely fast JSON serialisation (System.Text.Json), outperforming platforms like Node.js and Python under heavy API concurrency.
- **Structuring for Large Scale:** Controller-based APIs enforce strict class-based organisation, file separation, constructor injection, and attribute-based filter pipelines out of the box. This provides clear boundaries that help you write maintainable code for larger projects.
- **Strong Type Safety & Enterprise Standards:** Compile-time type checking, explicit DTO contracts, and native Dependency Injection prevent runtime errors early, enforcing clean architecture patterns used in large-scale enterprise systems.
- **Built-in Native OpenAPI (Scalar) Support:** .NET 10 includes native OpenAPI integration, making it effortless to generate interactive documentation and API specs for frontend teams (such as React developers) to consume.
- **Hybrid Enterprise Architectures:** In enterprise environments, hybrid setups are common, for example, to run a traditional .NET MVC Razor setup for internal admin panels while serving React client frontends through `[ApiController]` endpoints within the same solution.

Setting up backend API

Creating the Project

Create a new .NET project using the webapi template, explicitly enabling controller support, and name it api:

```
dotnet new webapi --use-controllers -n api
```

Important Note (.NET 10 Change): In .NET 10, running dotnet new webapi defaults to **Minimal APIs** without a Controllers/ directory. Passing the --use-controllers flag forces the template to generate the traditional class-based [ApiController] folder structure needed for this course.

If multiple .NET SDK versions are installed, check them with:

```
dotnet --list-sdks
```

To lock the project to a specific SDK version, create a global.json file in the root folder:

```
dotnet new globaljson --sdk-version "10.0.400"
```

After this, the folder structure looks as below:



Transitioning Code: MVC Monolith to Decoupled API

Delete the Controllers folder and WeatherForecast.cs in the default API project first.

Then copy the core business assets from our old MVC project to the api folder:

- **Controllers**
- **DAL (Data Access Layer / DbContext)**
- **Models**

Also copy (explained later):

- **ViewModels**
- **wwwroot/images**

Do **not** copy the **Views/** folder, an API returns structured data (JSON), not rendered HTML pages. Do **not** copy **wwwroot/libs/** folder.

Educational Context vs. Production Standards:
In this project, API Controllers are coded alongside old MVC controllers to directly compare the two architectures side-by-side. As a result, **ViewModels** and **wwwroot** are temporarily retained. In a professional setting, you would create a clean API project and use **DTOs (Data Transfer Objects)** instead of **ViewModels** to clearly define your data contracts and avoid mixing architectural patterns. In this demo, we keep both as a temporary solution so that you can easily compare the two architectures.

After copying, we get these:



Database Context & Lazy Loading Setup
Verify ItemDbContext.cs. Ensure it inherits from DbContext rather than IdentityDbContext if it only manages domain data (e.g., items) rather than user authentication identities:

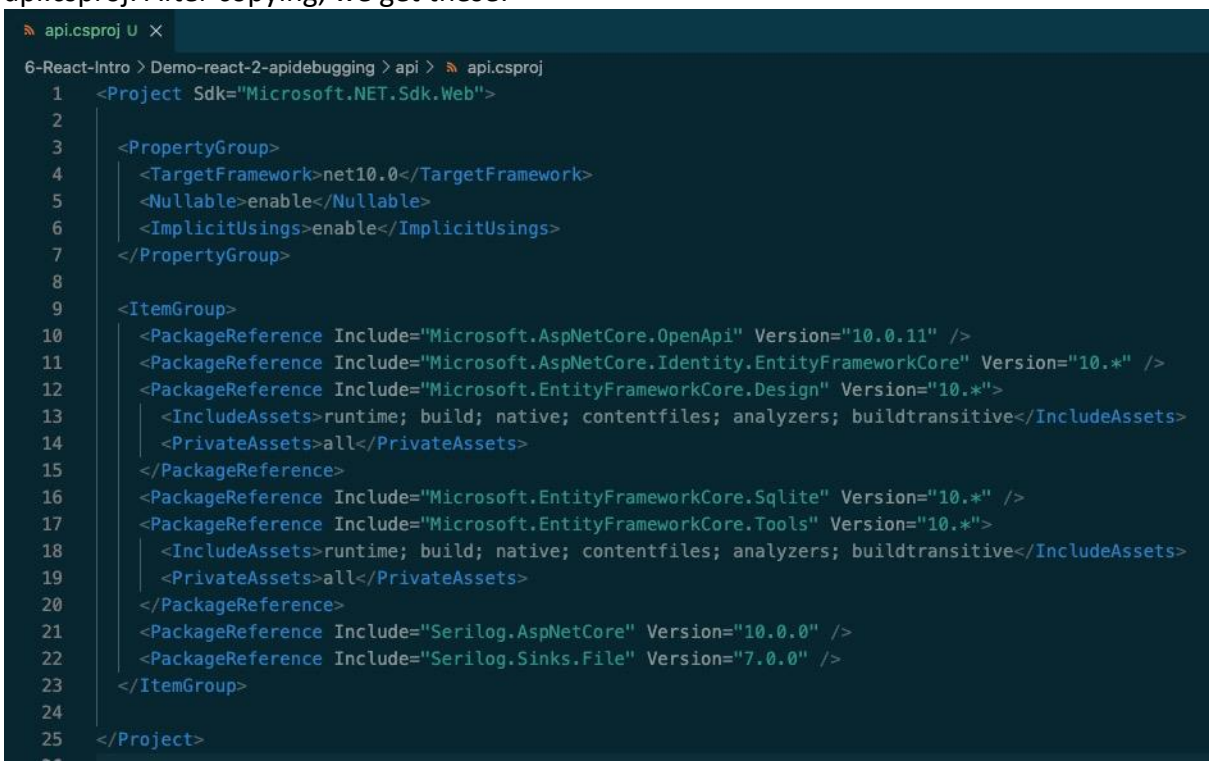
Remove Lazy Loading (UseLazyLoadingProxies)
Remove optionsBuilder.UseLazyLoadingProxies() from ItemDbContext.cs.
Note: You do **not** need to remove the virtual keyword from your models. Without the proxy builder enabled, EF Core simply treats virtual navigation properties as standard properties.
Why Lazy Loading is an Anti-Pattern for APIs

- **The N+1 Query Disaster:** During JSON serialisation, the serialiser accesses every property via reflection. With lazy loading enabled, each navigation property triggers a separate, synchronous SQL query during serialisation, resulting in excessive hidden database calls for a single request.
- **Data Bloat & Exposure:** Lazy loading recursively walks down navigation properties, serializing almost your entire database graph into memory and sending unwanted or sensitive relational data to the React client.



Managing Dependencies (.csproj)

From the csproj file of your backend, copy the needed PackageReference into the new api.csproj. After copying, we get these:



This dependency set includes only the essentials required for database management, user authentication, structured logging, and modern API documentation.

- **Microsoft.AspNetCore.OpenApi**: Retained to provide native metadata generation for API endpoints, making it easy to generate OpenAPI/Scalar documentation for React developers.
- **Microsoft.EntityFrameworkCore.Sqlite**: Retained as the database provider powering Entity Framework Core operations.
- **Microsoft.EntityFrameworkCore.Design & Tools**: Retained to execute database migrations and CLI commands via dotnet ef.
- **Microsoft.AspNetCore.Identity.EntityFrameworkCore**: Retained to manage user authentication, roles, and password hashing at the database layer (later).
- **Serilog.AspNetCore & Serilog.Sinks.File**: Retained to support structured file logging (such as saving logs to APILogs/).

Monolith-specific packages like **Microsoft.AspNetCore.Identity.UI** and **Microsoft.Web.LibraryManager.Build** were removed because React handles client-side rendering and package management. Additionally, **Microsoft.EntityFrameworkCore.Proxies** was removed to eliminate lazy loading, preventing N+1 query performance issues during JSON serialisation.

`Program.cs` Pipeline Configuration

Modify the Program.cs to this:

Program.cs U

6-React-Intro > Demo-react-2-apidebugging > api > Program.cs

```
using Microsoft.EntityFrameworkCore;
using MyShop.DAL;
using Scalar.AspNetCore;
using Serilog;
using Serilog.Events;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers().AddJsonOptions(options =>
    {
        options.JsonSerializerOptions.ReferenceHandler = // Prevents circular reference exceptions
            System.Text.Json.Serialization.ReferenceHandler.IgnoreCycles;
    });

builder.Services.AddOpenApi(); // 2. Native .NET OpenAPI Support

builder.Services.AddDbContext<ItemDbContext>(options =>
{
    options.UseSqlite(builder.Configuration.GetConnectionString("ItemDbContextConnection"));
});

builder.Services.AddCors(options => // CORS Policy for React Frontend Integration
{
    options.AddPolicy("CorsPolicy", policy =>
        policy.SetIsOriginAllowed(origin => new Uri(origin).Host == "localhost") // Allows any port on localhost
            .AllowAnyMethod().AllowAnyHeader().AllowCredentials());
});

builder.Services.AddScoped<IItemRepository, ItemRepository>();

var loggerConfiguration = new LoggerConfiguration()
    .MinimumLevel.Warning()
    .WriteTo.File($"Logs/api_{DateTime.Now:yyyyMMdd_HHmmss}.log");

var logger = loggerConfiguration.CreateLogger();
builder.Logging.AddSerilog(logger);

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    DBInit.Seed(app);
    app.MapOpenApi(); // Serve modern OpenAPI document endpoint (/openapi/v1.json)
    app.MapScalarApiReference(); // Serve Scalar Interactive API Documentation UI (/scalar/v1)
}

app.UseCors("CorsPolicy");
app.MapStaticAssets();
app.MapControllers();

app.Run();
```

Explanations

- **builder.Services.AddControllers().AddJsonOptions(...):** Registers API controllers and configures .NET's high-performance native System.Text.Json serialiser to ignore cycle references.
    - **Why circular references are a problem:** Entity Framework navigation properties create bidirectional relationships (for example, an Order has a Customer, and that Customer holds a list of Orders). During JSON serialization, the serializer recursively reads every property down the graph. Without IgnoreCycles, this creates an infinite loop (Order → Customer → Order → Customer...), causing the

server to throw a JsonException ("object cycle detected") and crash the request. Ignoring cycles tells the serializer to stop following a navigation property if it encounters an entity, it has already processed.

- **builder.Services.AddOpenApi():** Registers native .NET OpenAPI metadata extraction services for endpoint generation.
- **builder.Services.AddCors(...):** Configures Cross-Origin Resource Sharing (CORS), allowing all localhost ports. For production, you would switch to explicit domain URLs like `policy.WithOrigins("[https://myfrontend.com](https://myfrontend.com)")`.
- **Serilog Setup:** Configures file logging to automatically dump structured logs into the Logs/ directory while filtering out routine SQL query execution logs to reduce noise. We set the `.MinimumLevel.Warning()` to reduce log noise by suppressing routine EF Core database queries and internal HTTP request logs. It ensures `Logs/` files only capture actual warnings and application exceptions.
- **app.MapOpenApi() & app.MapScalarApiReference():**
    - app.MapOpenApi() exposes the raw OpenAPI JSON specification document at /openapi/v1.json.
    - app.MapScalarApiReference() generates an interactive, modern API documentation dashboard (accessible at /scalar/v1) for testing endpoints during development.
- **app.UseCors("CorsPolicy"):** Applies the CORS policy to incoming requests prior to reaching the API controllers.
- **app.MapStaticAssets():** Enables serving static files (e.g., product images inside wwwroot/images) directly to the React frontend. Endpoint mapping methods like `MapControllers()`, `MapOpenApi()`, and `MapStaticAssets()` should be placed at the end of the pipeline after all `app.Use...` middleware (like `UseCors`).
- **app.MapControllers():** Enables attribute-based routing ([ApiController], [Route("api/[controller]")]), directly mapping HTTP requests to controller action methods.

Scalar for debugging
run the following command in your terminal inside the api project directory:
`dotnet add package Scalar.AspNetCore`

Scalar is an interactive OpenAPI UI package built specifically to work with this native metadata generator.

Adding Scalar.AspNetCore automatically adds the required assembly reference so that using Scalar.AspNetCore; and app.MapScalarApiReference() resolve correctly in Program.cs.

Configuration Setup

Ensure appsettings.json contains the database connection string:

appsettings.json

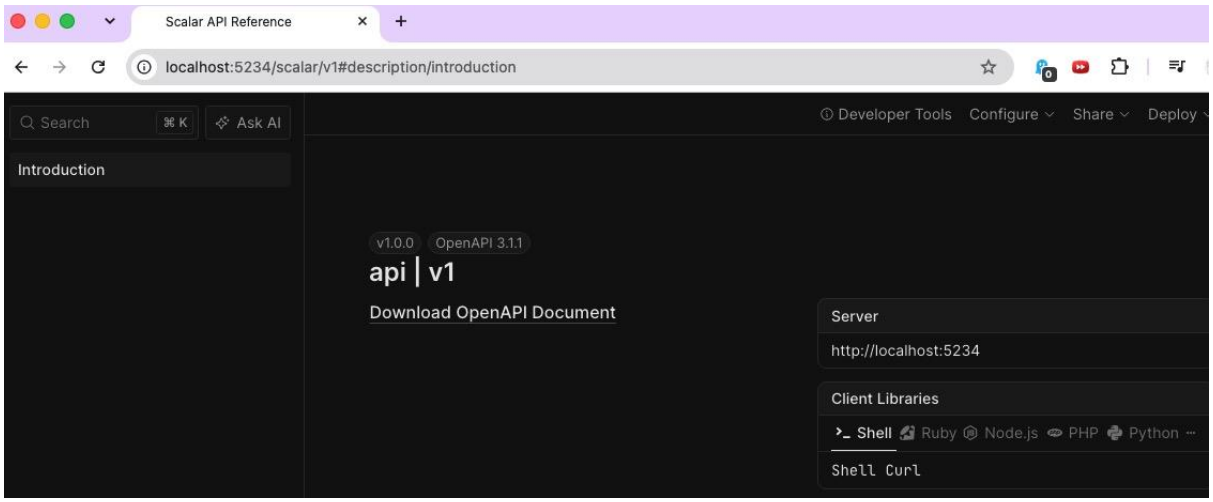
api > appsettings.json > ...

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "ItemDbContextConnection": "Data Source=ItemDatabase.db"
  }
}
```

Test the api with scalar

Run dotnet run in the api folder, then go to
`http://localhost:yourport/scalar/v1`
for me it is http://localhost:5234/scalar/v1

Then you can see that the scalar UI for debugging API is running. Currently it is empty because we do not have any API endpoints yet.



Building Web API Endpoints with Data Transfer Objects

When building a Web API, **we do not expose internal database entities directly to the outside world.** Instead, Web APIs use **Data Transfer Objects (DTOs)** to define the exact shape of data sent and received over HTTP.

Why We Use DTOs Instead of Database Entities

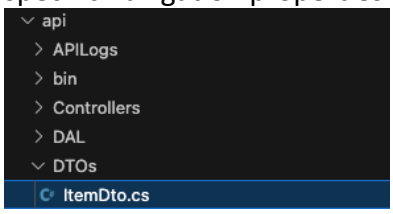
- **Security (Preventing Mass Assignment / Over-Posting):** DTOs allow you to control and limit what data is exposed to clients
    - In monolithic MVC, passing a Domain.Item entity to a Razor View is safe because C# runs on the server. If your Item entity has properties like SupplierCost, CreatedByAdminId, or InternalNotes, Razor only renders the HTML tags you explicitly write (<h1>@Model.Name</h1>). The extra properties remain inside server memory and are destroyed when the HTTP request finishes.
    - In a Web API, returning return Ok(item) serialises the entire C# object into JSON and sends it over the public network. Anyone can press **F12** in their browser, inspect the Network tab, and see internal or sensitive entity fields—even if the React UI never renders them.
- **API Contract Stability:** DTOs help separate the data structure used by the API from the internal domain model. This makes your API more adaptable to changes in the internal domain model and vice versa. DTOs allow you to tailor the data format and structure to meet the needs of different clients and API consumers.
    - In monolithic MVC, the backend and frontend are built and deployed together as a single C# application. If you rename a database column from ItemName to Name, you update your C# entity, update your Razor View, and deploy. Nothing breaks.
    - In a React + API setup, your API is a distributed data service that can be used by different data consumers. You can have different versions of frontend applications (some users may forget to update their frontend) running in their browsers, or phones. If you change a domain entity property directly, you immediately break every client application running the old code. DTOs insulate external clients from internal database refactoring.
- **Payload Optimisation:** Database entities contain navigation properties (e.g., Item.Category, Item.Orders) that create bloated payloads or deep relational trees in JSON responses. DTOs flatten and tailor data specifically to what the user interface needs.
- **Clean OpenAPI / Scalar Documentation:** Scalar generates documentation directly from your API models. Using clean DTOs ensures client developers see clear, simple JSON request and response schemas.

Create the Data Transfer Object (ItemDto)

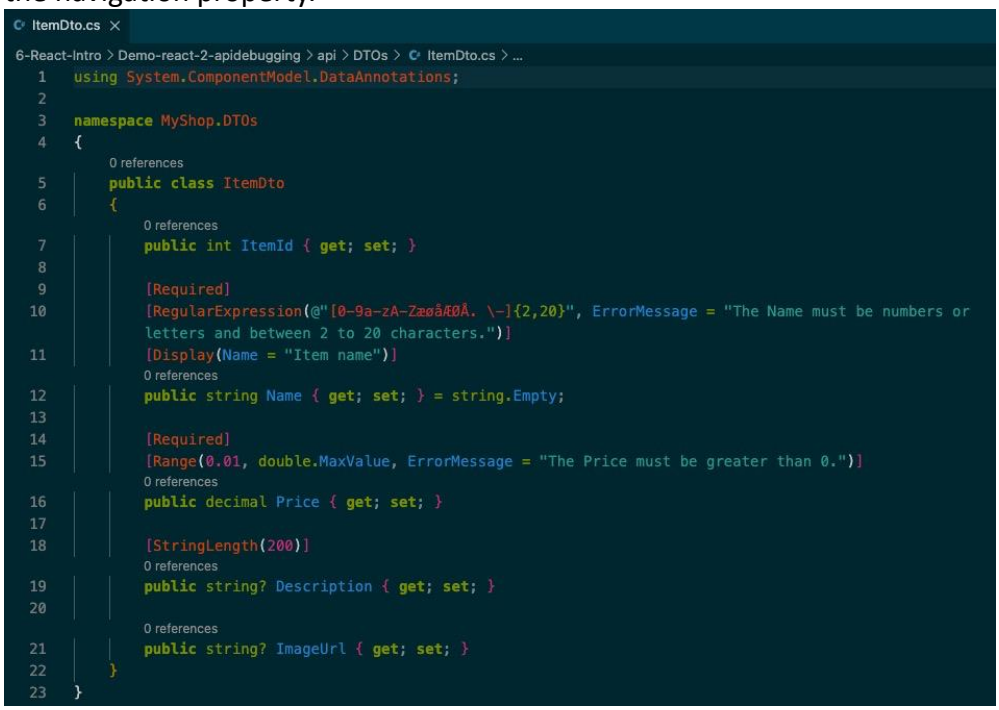
We start by defining the public contract for our item data.

- Create a folder named DTOs inside your API project.
- Inside DTOs/, create a file named ItemDto.cs

Dtos are a bit similar to ViewModels in idea, data for the user view). `ItemDto` mirrors the properties of an item that clients need to display, but intentionally leaves out database-specific navigation properties (such as `Orders` or `Category`).

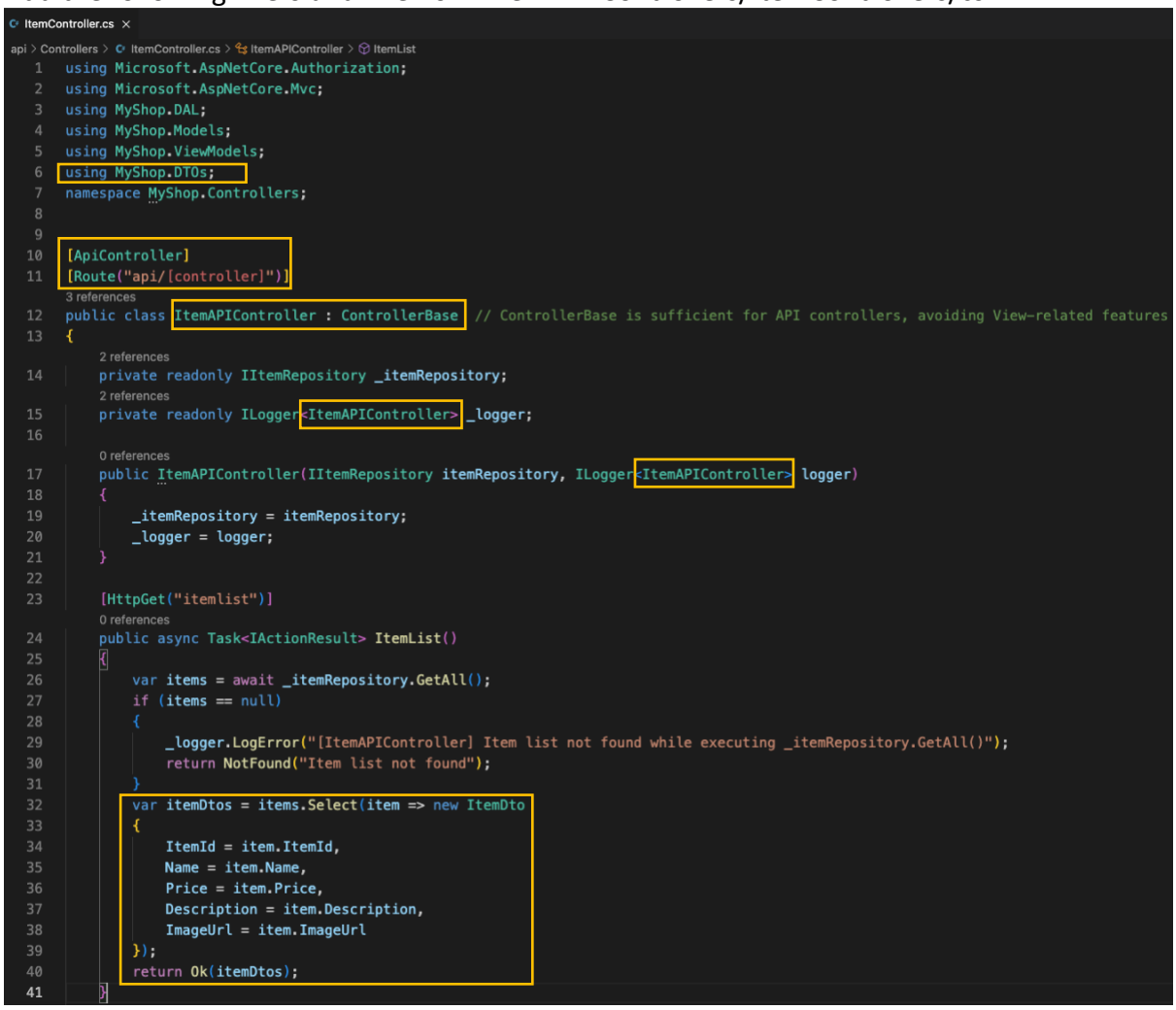


We create a file ItemDto.cs under DTOs. It is essential the same as the Item.cs, but without the navigation property.



Create the API Controller

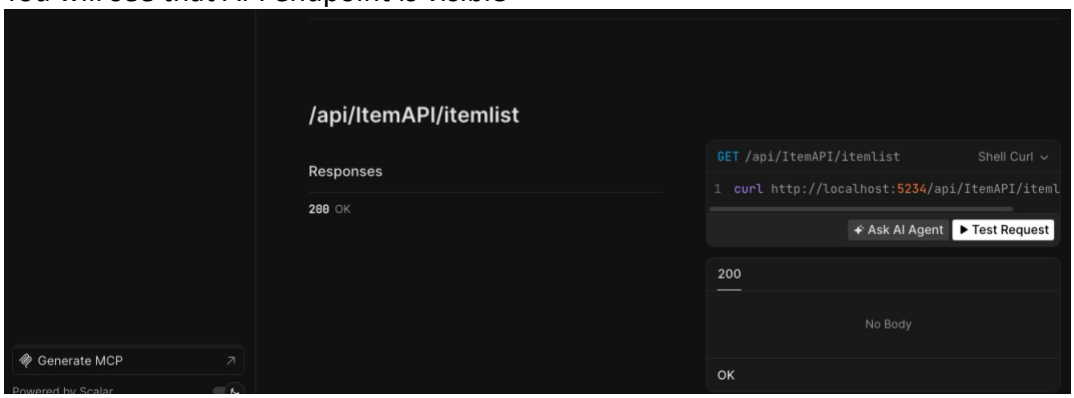
Now, build the controller and map domain entities retrieved from the repository into ItemDto objects before sending the HTTP response.
Add the following line 6 and line 10 – line 41 in Controllers/ItemControllers/cs:



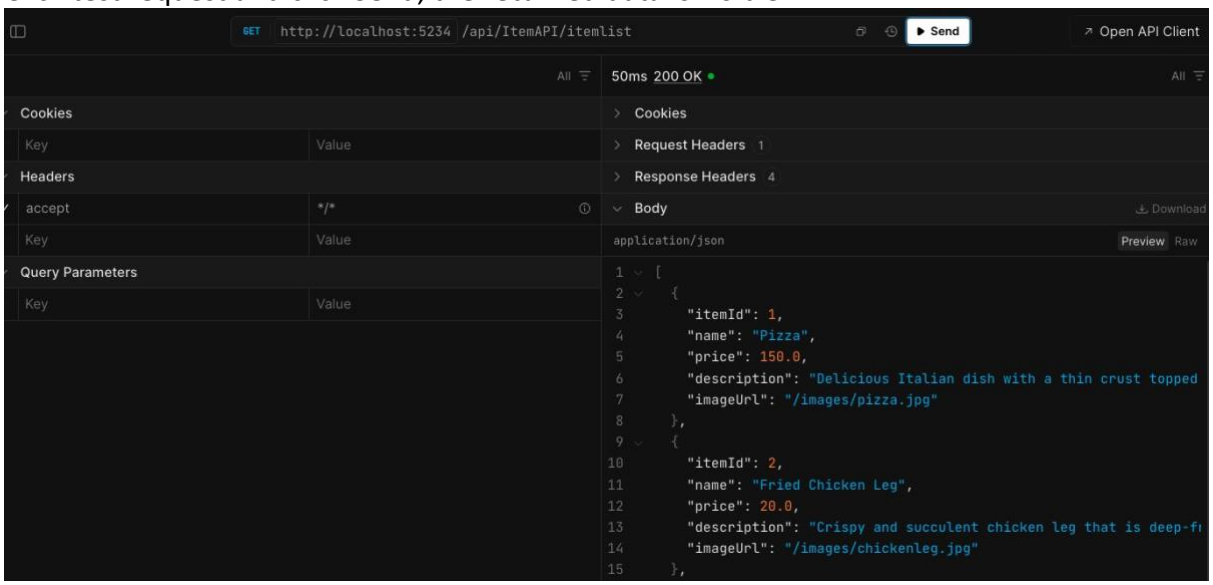
[ApiController]: This attribute marks the class as an API controller. It provides automatic model validation, error handling, and binding for API requests.
[Route("api/[controller]")]: This attribute defines the base route for all actions in this controller. [controller] is a placeholder that will be replaced with the name of the controller (minus the "Controller" suffix). For this controller named ItemAPIController, the base route will be api/itemapi.
[HttpGet("itemlist")]: This attribute specifies that the ItemList method should handle HTTP GET requests at the route api/itemapi/itemlist. It maps the GET request to this method.
public async Task<IActionResult> ItemList(): This is an asynchronous method that returns an IActionResult. The async keyword indicates that the method will perform asynchronous operations.
if (items == null): Checks if the retrieved items are null. If so, it logs an error message and returns a 404 Not Found response with the message "Item list not found".
.Select(item => new ItemDto { ... }): Projects internal entity records into public ItemDto instances, stripping internal state before returning data to the client.
return Ok(itemDtos): Wraps the DTO collection in an HTTP 200 OK response.

Test the api endpint

Now run the app with dotnet run and go to
`http://localhost:yourport/scalar/v1`
for me it is http://localhost:5234/scalar/v1
You will see that API endpoint is visible



Click test request and click Send, the returned data is visible:

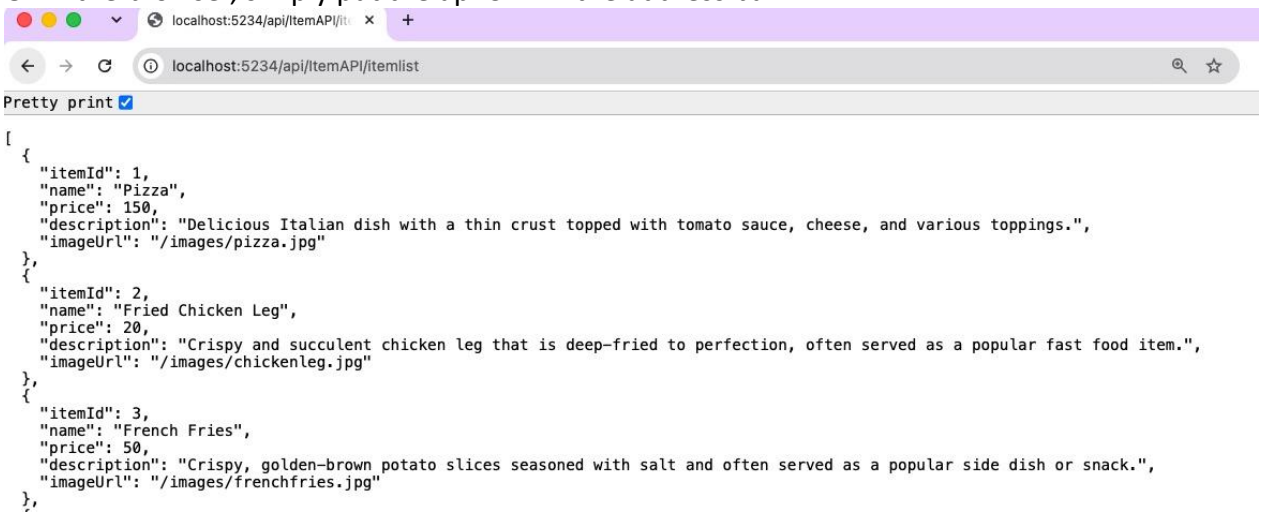


Alternative ways of testing the backend API

There are other ways to test the backend without using Scalar.
In the terminal, using the curl command:
curl http://localhost:5234/api/itemapi/itemlist

```
[USER]:ITPE3200-26H % curl http://localhost:5234/api/itemapi/itemlist
[{"itemId":1,"name":"Pizza","price":150.0,"description":"Delicious Italian dish with a thin crust topped
with tomato sauce, cheese, and various toppings.","imageUrl":"/images/pizza.jpg"},{"itemId":2,"name":"Fri
ed Chicken Leg","price":20.0,"description":"Crispy and succulent chicken leg that is deep-fried to perfec
tion, often served as a popular fast food item.","imageUrl":"/images/chickenleg.jpg"},{"itemId":3,"name":
"French Fries","price":50.0,"description":"Crispy, golden-brown potato slices seasoned with salt and ofte
n served as a popular side dish or snack.","imageUrl":"/images/frenchfries.jpg"},{"itemId":4,"name":"Gril
led Ribs","price":250.0,"description":"Tender and flavorful ribs grilled to perfection, usually served wi
th barbecue sauce.","imageUrl":"/images/ribs.jpg"},{"itemId":5,"name":"Tacos","price":150.0,"description"
:"Tortillas filled with various ingredients such as seasoned meat, vegetables, and salsa, folded into a d
elicious handheld meal.","imageUrl":"/images/tacos.jpg"},{"itemId":6,"name":"Fish and Chips","price":180.
0,"description":"Classic British dish featuring battered and deep-fried fish served with thick-cut fried
potatoes.","imageUrl":"/images/fishandchips.jpg"},{"itemId":7,"name":"Cider","price":50.0,"description":"
Refreshing alcoholic beverage made from fermented apple juice, available in various flavors.","imageUrl":
"/images/cider.jpg"},{"itemId":8,"name":"Coke","price":30.0,"description":"Popular carbonated soft drink
known for its sweet and refreshing taste.","imageUrl":"/images/coke.jpg"}]%
```

Or in the browser, simply put the api URL in the address bar:



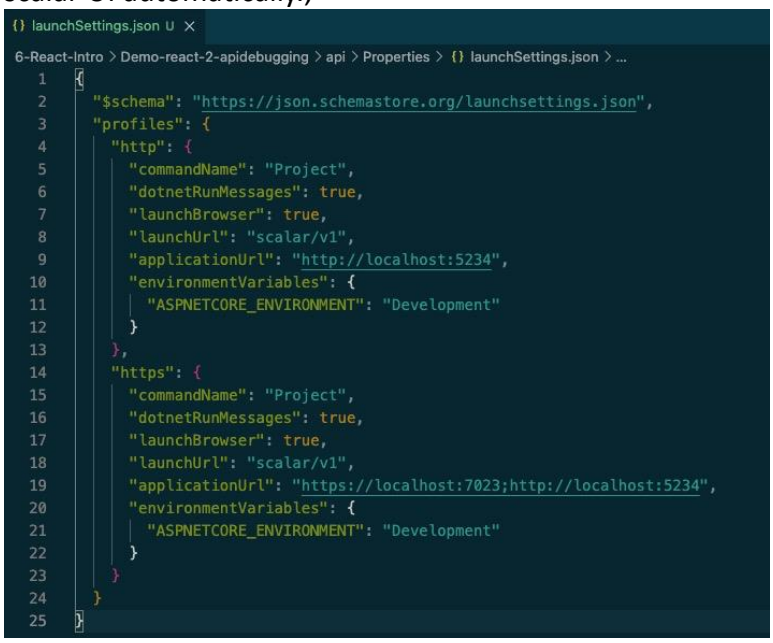
What is api.http for?
It is a built-in text file for testing endpoints directly inside Visual Studio or VS Code (similar to a lightweight Postman script). It allows you to click a "Send Request" button above an HTTP call and see the raw JSON response right inside your editor window. This remains to be explored by you.

Since we are already using **Scalar**, api.http is redundant. Scalar automatically discovers every new API endpoint you write via OpenAPI metadata, requiring zero manual file updates. We can safely deleting api.http from the project.

Setting up debugging of the backend API

Configuring Properties/launchSettings.json and dotnet watch

Open api/Properties/launchSettings.json and adjust the configuration as below. After that, you can run `dotnet watch` under the api folder, the browser will be opened and go to the scalar UI automatically.,

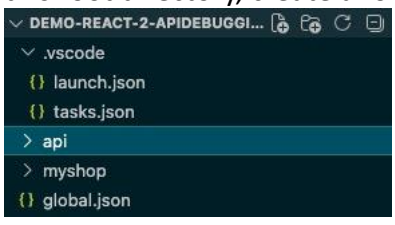


Key Settings:

- **launchBrowser: true**: Automatically opens your default web browser when debugging starts.
- **launchUrl: "scalar/v1"**: Navigates directly to the Scalar API Interactive Documentation dashboard.
- **ASPNETCORE_ENVIRONMENT: "Development"**: Ensures app.Environment.IsDevelopment() evaluates to true in Program.cs, enabling database seeding (DBInit.Seed), app.MapOpenApi(), and Scalar UI.

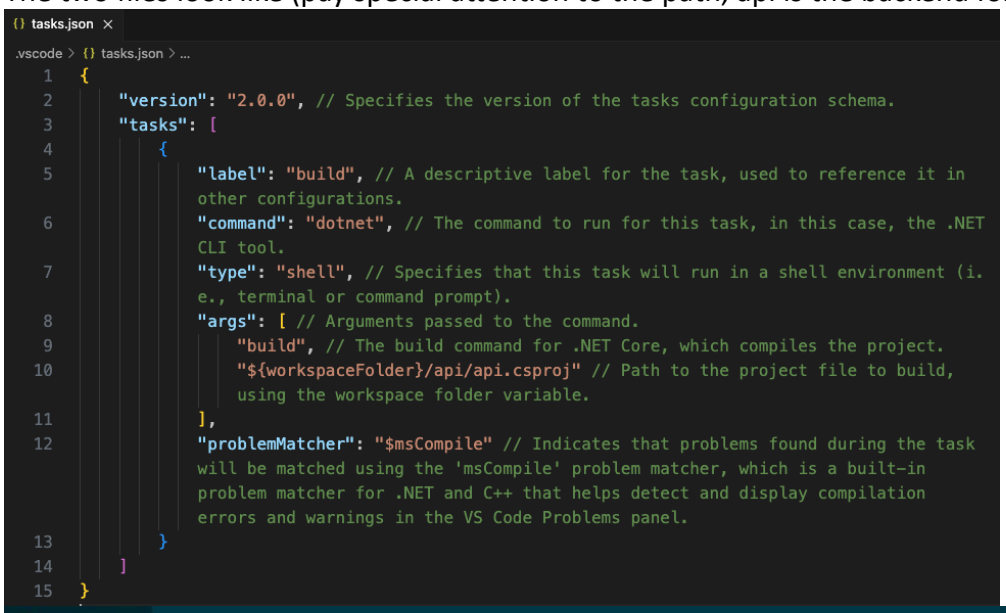
Configure VS Code Debugger (`.vscode/`)

Navigate to the parent folder containing both your api and myshop project directories. In this root directory, create a folder named .vscode.

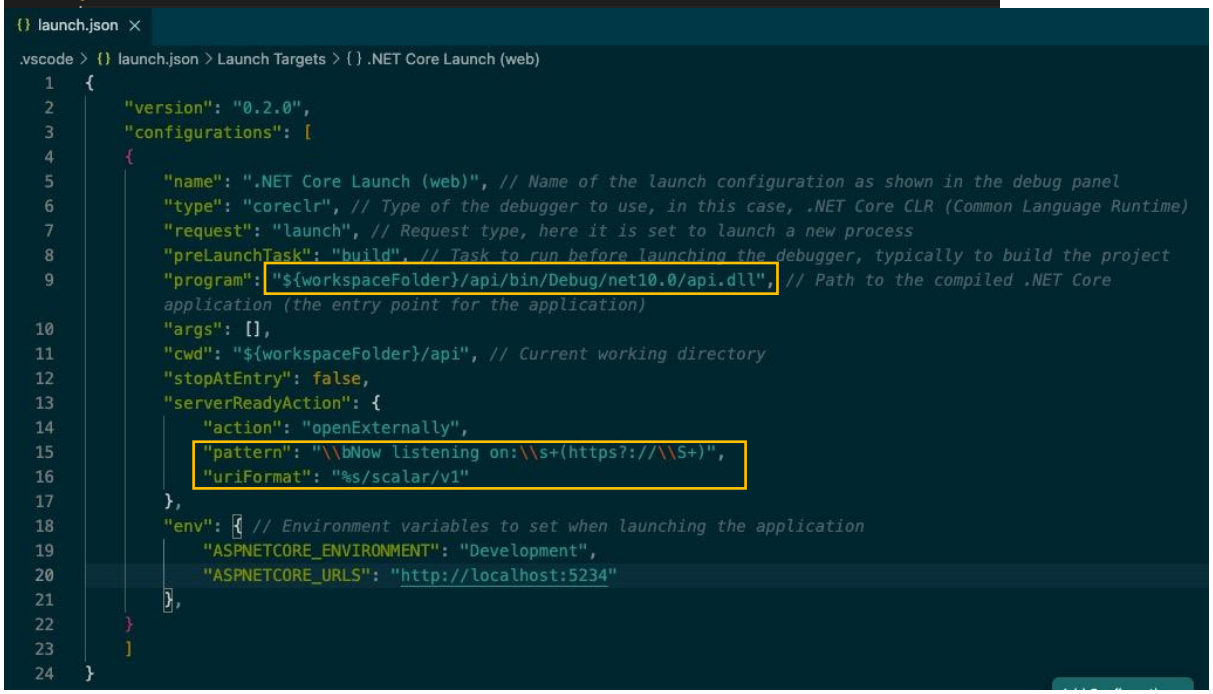


Inside .vscode, create two configuration files: launch.json and tasks.json.
When creating **launch.json**, you could click `create a launch.json file`, then select `C#`, then click `Add Configuration` (at right bottom corner, it normally auto pops up), then `.NET: Launch Executable File (Web)`.

The two files look like (pay special attention to the path, api is the backend folder):



```jsonc
{
    "version": "2.0.0", // Specifies the version of the tasks configuration schema.
    "tasks": [
        {
            "label": "build", // A descriptive label for the task, used to reference it in other configurations.
            "command": "dotnet", // The command to run for this task, in this case, the .NET CLI tool.
            "type": "shell", // Specifies that this task will run in a shell environment (i.e., terminal or command prompt).
            "args": [ // Arguments passed to the command.
                "build", // The build command for .NET Core, which compiles the project.
                "${workspaceFolder}/api/api.csproj" // Path to the project file to build, using the workspace folder variable.
            ],
            "problemMatcher": "$msCompile" // Indicates that problems found during the task will be matched using the 'msCompile' problem matcher, which is a built-in problem matcher for .NET and C++ that helps detect and display compilation errors and warnings in the VS Code Problems panel.
        }
    ]
}
```



```jsonc
{
    "version": "0.2.0",
    "configurations": [
    {
        "name": ".NET Core Launch (web)", // Name of the launch configuration as shown in the debug panel
        "type": "coreclr", // Type of the debugger to use, in this case, .NET Core CLR (Common Language Runtime)
        "request": "launch", // Request type, here it is set to launch a new process
        "preLaunchTask": "build", // Task to run before launching the debugger, typically to build the project
        "program": "${workspaceFolder}/api/bin/Debug/net10.0/api.dll", // Path to the compiled .NET Core application (the entry point for the application)
        "args": [],
        "cwd": "${workspaceFolder}/api", // Current working directory
        "stopAtEntry": false,
        "serverReadyAction": {
            "action": "openExternally",
            "pattern": "\\bNow listening on:\\s+(https?://\\S+)",
            "uriFormat": "%s/scalar/v1"
        },
        "env": { // Environment variables to set when launching the application
            "ASPNETCORE_ENVIRONMENT": "Development",
            "ASPNETCORE_URLS": "http://localhost:5234"
        },
    }
    ]
}
```

Now run the project in debugging mode by pressing F5.
It should automatically open the browser and go to http://<your-url>/scalar/v1 . If not, you could open the browser, go to the address: http://localhost:5234/scalar/v1
(replace it with your URL in the api/Properties/launchSettings.json)

Congratulations! Now you have a running API and learnt how to test and debug it! ☺